

Lab 02 — Images, lagen en registries

Theorie — hoe een image benoemd wordt, waaruit ze bestaat, waar ze vandaan komt, en waarom Podman weigert te gokken.

Doelstellingen

- Een volledige imagenaam ontleden en weten wat Docker impliciet aanvult — en wat Podman weigert aan te vullen.
- Begrijpen waarom een **tag** een bewegend label is en een **digest** een identiteit.
- Het **lagenmodel** uitleggen en wat het betekent voor schijf en netwerk.
- Een image kunnen inspecteren zonder ze te starten.
- Docker Hub, private registries en "officiële" images kunnen plaatsen.

1. De volledige naam van een image

Jij schrijft `podman pull postgres:16-alpine`. De engine begrijpt:

```
docker.io / library / postgres : 16-alpine
└registry┐ └namespace┐ └repo┐ └tag┐
```

Deel	Rol	Standaardwaarde
registry	De server die de image host	<code>docker.io</code> (Docker Hub)
namespace	De organisatie of gebruiker die eigenaar is	<code>library</code> (officiële images)
repository	De naam van de applicatie	<i>verplicht</i>
tag	De versie	<code>latest</code>

Drie onmiddellijke gevolgen:

- `postgres` alleen betekent `docker.io/library/postgres:latest`.
- Een bedrijfsimage draagt een volledige naam: `registry.mijnbedrijf.be/team-betalingen/api-facturatie:1.4.2`. Een **punt** of een **poort** in het eerste deel wijst op een registry en niet op een namespace.
- De images van de namespace `library` zijn de **officiële images**: onderhouden samen met Docker, geauditeerd, gedocumenteerd (`postgres`, `nginx`, `node`, `eclipse-temurin`). Een image `bobvan59/postgres` heeft geen enkele van die garanties.

PODMAN

Docker vult `postgres` aan tot `docker.io/library/postgres` zonder iets te zeggen. Podman ziet daar een risico in: een korte naam kan naar een andere image verwijzen naargelang de bevraagde registry (*typosquatting*, naamgenoot op een interne registry). Het past `registries.conf` toe: gekende **aliases** (`alpine`, `nginx`, `postgres` ...) die zonder vraag worden opgelost, en voor de rest de lijst `unqualified-search-registries` — zijn er meerdere, dan vraagt het je te kiezen. Op Ubuntu bevat die lijst alleen `docker.io`, dus korte namen "werken"; op Fedora of een `podman machine` lokken ze een vraag uit. Een lokaal gebouwde image krijgt het voorvoegsel `localhost/`: `podman build -t api:1.0` levert `localhost/api:1.0`. `podman images` toont altijd de volledige naam, zonder magie.

VALKUIL

`latest` betekent niet "de laatste versie". Het is een **standaard**-tag zoals een andere, die de uitgever (al dan niet) verplaatst; hij kan naar een versie van twee jaar oud wijzen. In productie is `latest` verboden: niet-reproduceerbare uitrol, geen rollback mogelijk.

2. Bewegende tag, onveranderlijke digest

Een **tag** is een pointer: `postgres:16` betekent vandaag `16.10`, morgen `16.11`. Op je schijf beweegt niets, maar een nieuwe `pull` brengt iets anders. Een **digest** is de SHA-256-vingerafdruk van het manifest: `postgres@sha256:9d0d1f1e...`. Hij wordt **berekend uit de inhoud**, dus:

- twee images met dezelfde digest zijn bit voor bit identiek, waar ze ook staan;
- een image kan niet veranderen zonder dat haar digest verandert;
- een uitrol kan dus absoluut vastgepind worden.

→ BEVEILIGING

SHA-256 is een **hashfunctie**: ze zet eender welke data om in 64 hexadecimale tekens, deterministisch (zelfde invoer, zelfde uitvoer) en in één richting (onmogelijk een invoer te maken die een gekozen uitvoer oplevert). Eén bit van de image wijzigen verandert de hele vingerafdruk. Het is hetzelfde principe dat een Git-commit identificeert: een identificatie die *tegelijk* een integriteitsbewijs is.

```
podman image inspect --format '{{.Digest}}' postgres:16-alpine
podman pull docker.io/library/postgres@sha256:9d0d1f1e... # perfect reproduceerbaar
```

Het gebruikelijke compromis in bedrijven: één unieke, onveranderlijke tag per build (`api:1.4.2` of `api:2026.03.17-b318`), nooit hergebruikt, en uitroltools die de digest vastpinnen.

3. Een image is een stapel lagen

Elke bouw instructie die het bestandssysteem wijzigt, produceert een **laag** (*layer*): een verzameling bestanden die toegevoegd, gewijzigd of verwijderd zijn ten opzichte van de vorige toestand. De uiteindelijke image is de superpositie van die lagen, plus een manifest dat ze oplijst en een configuratie (standaardcommando, variabelen, gebruiker...).

	laag 4: COPY app.jar	(60 MB)
	laag 3: de JRE	(180 MB)
	laag 2: systeempakketten	(30 MB)
	laag 1: basis Debian slim	(75 MB)

↑ alleen-lezen, gedeeld door alle images die ze bevatten

→ LINUX

Het opslagstuurprogramma `overlay` is een kernelbestandssysteem dat mappen **stapelt**: "lagere" alleen-lezen lagen en één "hogere" schrijflaag. Lezen zoekt van boven naar onder; schrijven kopieert het bestand eerst naar de hogere laag (*copy-on-write*); verwijderen maakt een spookbestand (*whiteout*) dat verbergt zonder weg te nemen. Het hele gedrag van images volgt uit die drie regels.

Die structuur verklaart vier gedragingen die je voortdurend zult zien:

1. Delen op schijf. Als je twaalf microservices van dezelfde JRE vertrekken, worden die 180 MB **één keer** opgeslagen. De som van de kolom `SIZE` van `podman images` overstijgt dus ruim de gebruikte ruimte — `podman system df` geeft het echte cijfer.

2. Differentiële overdracht. Een `pull` of `push` draagt alleen de ontbrekende lagen over: je API opnieuw uitrollen draagt vaak alleen de JAR-laag over.

3. Onveranderlijkheid, fouten inbegrepen. Als een laag een wachtwoord toevoegt en een latere laag het verwijdert, **zit het bestand nog altijd in de image**: de latere laag verbergt het alleen, en wie de image heeft, kan het terughalen. Een geheim mag nooit in een build terechtkomen (lab 08).

4. De buildcache. Omdat lagen op inhoud geïdentificeerd worden, hergebruikt de engine wat ze al heeft (lab 04).

ONTHOUDEN

Lagen worden gedeeld op schaal van de host of de registry, niet van de image: een identieke laag in twee images wordt één keer opgeslagen. In rootless-modus zit die opslag in **jouw** `home` (`~/.local/share/containers/storage`): twee gebruikers van dezelfde machine delen niets.

VALKUIL

Een tag verwijst eigenlijk naar een **manifest list**, één per architectuur (`linux/amd64`, `linux/arm64` ...); de `pull` kiest die van jouw machine. Een image gebouwd op een MacBook met Apple Silicon weigert dus te starten op een `amd64`-server: `exec format error`. `--platform` dwingt de architectuur af.

4. De dagelijkse commando's

```
podman pull nginx:alpine           # downloaden zonder starten
podman images                      # lokale images ophalen
podman images --filter dangling=true # images zonder tag (weeslagen)
podman history nginx:alpine         # de lagen, hun grootte en herkomst
podman image tree nginx:alpine      # de lagen... als boom, met hun bronimage
podman image inspect nginx:alpine   # volledige metadata als JSON
podman tag nginx:alpine mijn-nginx:v1 # een naam toevoegen (wordt localhost/mijn-nginx:v1)
podman rmi mijn-nginx:v1            # een naam verwijderen (en de image als het de laatste was)
podman system df                   # werkelijk gebruikte ruimte
```

Twee slecht begrepen subtiliteiten:

podman tag kopieert niets. Het voegt een label toe aan dezelfde image; beide namen verwijzen naar dezelfde `IMAGE ID`. Omgekeerd verwijdert `rmi` op een image met twee tags alleen de tag: de data verdwijnt pas als de laatste naam weg is.

Een "dangling" image (`<none>:<none>`) is geen mysterieus afval. Het is een image waarvan de tag naar een recentere versie verplaatst is: ze verloor haar naam maar neemt nog schijfruimte in — het normale residu van opeenvolgende rebuilds.

Een image uit de engine halen

```
podman save -o api.tar mijn-api:1.0 # archief in docker-archive-formaat
podman save --format oci-archive -o api.tar mijn-api:1.0 # hetzelfde, in het OCI-standaardformaat
podman load -i api.tar # importeert de image opnieuw, tags inbegrepen
```

Nuttig wanneer het doel geen toegang heeft tot de registry (geïsoleerde site). Niet te verwarren met `export` / `import`, die het bestandssysteem **van een container** platslaan en lagen en configuratie verliezen (`CMD`, `ENV`, `EXPOSE` ...).

5. De registries

Een registry is een HTTP-dienst die lagen en manifesten opslaat achter een gestandaardiseerde API (`/v2/...`) die alle tools spreken.

→ HTTP

Een REST-API stelt *resources* bloot op URL's en bewerkt ze met HTTP-werkwoorden: `GET /v2/_catalog` lijst de repositories op, `HEAD /v2/api/manifests/1.0` geeft de digest terug in een header, `PUT` pusht een laag. `curl` volstaat dus om een registry te bevragen — je doet het in het praktijklab. Het is de mechaniek van een Spring Boot-API.

Type	Voorbeelden	Gebruik
Publiek	Docker Hub, <code>ghcr.io</code> , <code>quay.io</code>	Basisimages en kant-en-klare software
Privaat, beheerd	AWS ECR, Azure ACR, Google AR	Eigen images, gehost bij de cloud
Privaat, zelf gehost	Harbor, Nexus, GitLab Registry	Eigen images, volledige controle, kwetsbaarheidsscan

De cyclus in het bedrijf:

```
podman login registry.mijnbedrijf.be
podman tag api:1.4.2 registry.mijnbedrijf.be/betalingen/api:1.4.2
podman push registry.mijnbedrijf.be/betalingen/api:1.4.2
```

Drie dingen om te weten:

- `podman login` **bewaart het token** in `${XDG_RUNTIME_DIR}/containers/auth.json`, een tijdelijk bestand dat bij het afmelden gewist wordt — waar Docker in `~/.docker/config.json` schrijft, leesbaar (base64) en voor altijd. Op een CI-agent verkiest men kortstondige credentials.
- **Podman eist TLS.** Een registry in gewone HTTP — zoals die welke je op `localhost:5000` zult starten — wordt geweigerd (`http: server gave HTTP response to HTTPS client`) zolang je niet `--tls-verify=false` zegt of de registry `insecure = true` verklaart in `registries.conf`. Docker maakt stilzwijgend een uitzondering voor `localhost`; Podman niet.
- **Docker Hub beperkt anonieme downloads** (quotum per IP): op een CI geeft dat `toomanyrequests`, vandaar het gebruik van een *pull-through cache* of een interne kopie van de basisimages.

6. In het bedrijf

Op een Spring Boot + Angular-stack:

- De **basisimages** (`eclipse-temurin` , `node` , `nginx` , `postgres`) worden gekopieerd naar de interne registry, vaak met `skopeo copy` — de zuster tool van Podman die van de ene registry naar de andere kopieert zonder iets te downloaden. Niemand haalt in productie iets van het internet: quotum, beschikbaarheid, controle over wat binnenkomt.
 - De CI bouwt `registry.intern/mijnapp/api:<versie>` en `.../web:<versie>` , en pusht; de versie komt van de Git-tag of het buildnummer. Een **scanner** (Trivy, Harbor, Gype) blokkeert images met kritieke kwetsbaarheden. De uitrol verwijst naar een precieze versie, nooit `latest` .
-

Onthouden

- Een volledige naam is `registry/namespace/repository:tag` ; standaard `docker.io` , `library` en `latest` . Podman toont altijd die volledige naam en zet `localhost/` voor je eigen builds.
- `latest` is niet "de recentste": het is een standaard-tag, te bannen in productie.
- De **tag** kan bewegen, de **digest** `sha256:...` identificeert een exacte inhoud.
- Een image is een stapel **alleen-lezen lagen**, gedeeld tussen images, differentieel overgedragen — en een bestand dat in een latere laag verwijderd is, blijft erin zitten: nooit een geheim in een build.
- `tag` dupliceert niets; `rmi` verwijdert eerst een naam, geen data. Podman eist TLS: `--tls-verify=false` alleen voor een lokale testregistry.
- `save` / `load` vervoeren een volledige image; `export` / `import` slaan een container plat en verliezen zijn configuratie.

Woordenschat

repository: de versies van een image. — **tag**: bewegend label. — **digest**: onveranderlijke SHA-256-vingerafdruk. — **manifest**: beschrijving van lagen en configuratie; de **manifest list** indexeert meerdere architecturen. — **dangling image**: image die haar tag verloor. — **layer**: bestandslaag. — **overlay**: stuurprogramma dat de lagen stapelt. — **pull-through cache**: lokale spiegel van een publieke registry. — **officiële image**: namespace `library` op Docker Hub. — **short name**: naam zonder registry, opgelost via `registries.conf` . — **skopeo**: images kopiëren en inspecteren tussen registries.